Use Cases

for

<Project>

Version 1.0 approved

Prepared by <author>

<organization>

<date created>

Revision History

Name	Date	Reason For Changes	Version

Guidance for Use Case Template

Document each use case using the template shown in the Appendix. This section provides a description of each section in the use case template.

1. Use Case Identification

1.1. Use Case ID

Give each use case a unique numeric identifier, in hierarchical form: X.Y. Related use cases can be grouped in the hierarchy. Functional requirements can be traced back to a labeled use case.

1.2. Use Case Name

State a concise, results-oriented name for the use case. These reflect the tasks the user needs to be able to accomplish using the system. Include an action verb and a noun. Some examples:

- View part number information.
- Manually mark hypertext source and establish link to target.
- Place an order for a CD with the updated software version.

1.3. Use Case History

1.3.1. Created By

Supply the name of the person who initially documented this use case.

1.3.2. Date Created

Enter the date on which the use case was initially documented.

1.3.3. Last Updated By

Supply the name of the person who performed the most recent update to the use case description.

1.3.4. Date Last Updated

Enter the date on which the use case was most recently updated.

2. Use Case Definition

2.1. Actor

An actor is a person or other entity external to the software system being specified who interacts with the system and performs use cases to accomplish tasks. Different actors often correspond to different user classes, or roles, identified from the customer community that will use the product. Name the actor(s) that will be performing this use case.

2.2. Description

Provide a brief description of the reason for and outcome of this use case, or a high-level description of the sequence of actions and the outcome of executing the use case.

2.3. Preconditions

List any activities that must take place, or any conditions that must be true, before the use case can be started. Number each precondition. Examples:

1. User's identity has been authenticated.
2. User's computer has sufficient free memory available to launch task.

2.4. Postconditions

Describe the state of the system at the conclusion of the use case execution. Number each postcondition. Examples:

1. Document contains only valid SGML tags.
2. Price of item in database has been updated with new value.

2.5. Priority

Indicate the relative priority of implementing the functionality required to allow this use case to be executed. The priority scheme used must be the same as that used in the software requirements specification.

2.6. Frequency of Use

Estimate the number of times this use case will be performed by the actors per some appropriate unit of time.

2.7. Flow of Events

Provide a detailed description of the user actions and system responses that will take place during execution of the use case under normal, expected conditions. This dialog sequence will ultimately lead to accomplishing the goal stated in the use case name and description. This description may be written as an answer to the hypothetical question, "How do I <accomplish the task stated in the use case name>?" This is best done as a numbered list of actions performed by the actor, alternating with responses provided by the system.

2.8. Alternative Flows

Document other, legitimate usage scenarios that can take place within this use case separately in this section. State the alternative course, and describe any differences in the sequence of steps that take place. Number each alternative course using the Use Case ID as a prefix, followed by "AC" to indicate "Alternative Course". Example: X.Y.AC.1.

2.9. Exceptions

Describe any anticipated error conditions that could occur during execution of the use case, and define how the system is to respond to those conditions. Also, describe how the system is to respond if the use case execution fails for some unanticipated reason. Number each exception using the Use Case ID as a prefix, followed by "EX" to indicate "Exception". Example: X.Y.EX.1.

2.10. Includes

List any other use cases that are included ("called") by this use case. Common functionality that appears in multiple use cases can be split out into a separate use case that is included by the ones that need that common functionality.

2.11. Special Requirements

Identify any additional requirements, such as nonfunctional requirements, for the use case that may need to be addressed during design or implementation. These may include performance requirements or other quality attributes.

2.12. Assumptions

List any assumptions that were made in the analysis that led to accepting this use case into the product description and writing the use case description.

2.13. Notes and Issues

List any additional comments about this use case or any remaining open issues or TBDs (To Be Determineds) that must be resolved. Identify who will resolve each issue, the due date, and what the resolution ultimately is.

Use Case Template

Use Case ID:	UC.1		
Use Case Name:	View Top Industries by Job Vacancies		
Created By:	Gan Sui	Last Updated By:	Gan Sui
Date Created:	6/2/26	Date Last Updated:	21/2/26

Actor:	Job seeker
Description:	This use case allows a Job Seeker to view a ranked dashboard of the Top 10 industries with the highest job vacancy counts based on the most recent quarter of vacancy data retrieved via API. The outcome is a clickable list of industries that the user can select to proceed to course recommendations (UC.2).
Preconditions:	1. The system has access to the job vacancy dataset via the government API. 2. The user has access to the website via a supported web browser (Chrome/Firefox/Safari).
Postconditions:	1. The system displays the Top 10 industries with: industry name, total vacancy count, and rank indicator. 2. The quarter/date of the job vacancy data used is displayed. 3. The system is ready for the user to select an industry (which triggers UC2).

Priority:	High
Frequency of Use:	High
Flow of Events:	1. The Job Seeker opens the website homepage and navigates to the Industry Dashboard. 2. The system retrieves the latest job vacancy dataset from the API. 3. The system selects the most recent quarter in the dataset and prepares the records for analysis. 4. The system removes aggregate categories (for example, “Total”, “Goods Producing Industries”, and “Services”) and excludes any industries with missing values marked as “na”. 5. The system totals the job vacancy counts by industry, ranks the industries, and determines the Top 10. 6. The system displays the Top 10 industries with their industry names, vacancy counts, and rank indicators, and also shows the quarter/date used. 7. The Job Seeker clicks an industry from the Top 10 list. 8. The system highlights the selected industry in light blue and proceeds to UC.2 (course retrieval for the selected industry).

Alternative Flows:	UC.1.AC.1: Fewer than 10 industries available 1. If fewer than 10 industries remain after excluding "na", displays all valid industries instead. UC.1.AC.2: User selects a different industry 1. The Job Seeker clicks a different industry from the Top 10 list after making an earlier selection. 2. The system removes the light blue highlight from the previously selected industry and applies the light blue highlight to the newly selected industry. 3. The system updates the selected industry state and initiates UC.2 to retrieve and display courses for the newly selected industry.
Exceptions:	UC.1.EX.1: Job vacancy data retrieved failure 1. The system displays an error message indicating that industry data is unavailable.

Includes:	UC.2: Retrieve & Explore SkillsFuture Courses for Selected Industry
Special Requirements:	1. The top industries dashboard must load within 5 seconds. 2. The system must function correctly on Chrome, Firefox, and Safari. 3. No personal user data shall be stored by the system. 4. The system must update CSV once a day.
Assumptions:	1. Job vacancy data is updated quarterly and and contains industry + vacancy count + quarter/date in a consistent structure. 2. Keyword matching provides a reasonable approximation of industry relevance.
Notes and Issues:	1. The accuracy of course recommendations depends on the quality of keyword matching. 2. Future improvements may include machine-learning-based course matching.